

SESSION 06 / 10

モデル読み込みと 外部リソース

06

外部でつくった3Dモデルを読み込み、自分のシーンに組み込む

Week 6 2 hours Three.js × WebGL 実践講座

本セッションの目的

外部でつくった3Dモデルを、自分のシーンで動かせるようにする

WHY

資産を活かす

3D制作ツールで作られたモデルを、自分で作り直さずに使う

WHAT

読み込んで整える

glTF/GLBを読み込み、位置とスケールをシーンに合わせる

NEXT

操作へつなげる

配置したモデルにクリック反応やアニメーションを加える

外部資産を読み込み・整え・組み込むことで、プロ品質の3Dシーンを自分の手で動かす

本日のアジェンダ

01

モデルを読み込む

GLTFLoader で glTF・GLB を読み込む

30 min

02

モデルを整える

スケール・中心・素材をシーンに合わせる

40 min

03

ハンズオン演習

モデルを配置してクリックで反応させる

50 min

解説 60 min → ハンズオン 50 min → 振り返り 10 min

前提知識・準備物

第5回の到達点

- ✓ OrbitControls で視点を操作できる
- ✓ Raycaster で対象を選択できる
- ✓ ポインタ座標を NDC に変換できる
- ✓ animate() の描画ループが回っている

今回必要なもの

前回のプロジェクト

my-threejs-app をそのまま使用

three/addons

GLTFLoader を import できる構成

サンプルモデル

.glb ファイルを1つ用意する

3Dモデルのファイル形式

モデルには複数の形式があり、用途に応じて使い分ける

OBJ

形状のみ

古くからある形式。テクスチャ・アニメは別管理

FBX

DCC間の定番

アニメ・階層を含むが仕様が複雑で重い

glTF / GLB

Web標準

軽量でThree.js公式対応。今回はこれを使う

この講座ではWeb配信に適した glTF / GLB を使う

読み込みの3ステップ

外部のモデルを取り込み、自分のシーンで扱えるようにするまでの流れ

LOAD

読み込む

GLTFLoader でファイルを取得し
gltf.scene を得る

ADJUST

整える

バウンディングボックスで中心とスケールをそろえる

INTEGRATE

組み込む

scene に追加し、操作やアニメを付ける

読み込む → 整える → 組み込む の流れで、つくった資産が動き出す

GLTFLoader — モデルを読み込む

glTF / GLB を読み込み、gltf.scene をシーンに追加する

```
import { GLTFLoader } from
  'three/addons/loaders/GLTFLoader.js';

const loader = new GLTFLoader();
loader.load('/models/robot.glb',
  (gltf) => {
    scene.add(gltf.scene); // 完了時
  });
```

load(url, onLoad)

読み込み完了時に呼ばれる

gltf.scene

中身の Group を取り出す

onProgress / onError

進捗と失敗を受け取る

LoadingManager — 進捗をまとめる

複数リソースの読み込み状況を1か所で受け取る

```
const manager =  
  new THREE.LoadingManager();  
manager.onProgress =  
  (url, loaded, total) => {  
    bar.width = loaded/total*100;  
  };  
manager.onLoad = () => hide(); // 全完了  
new GLTFLoader(manager);
```

onProgress

loaded / total で割合

onLoad

全リソース完了で発火

onError

失敗した URL を受け取る

DRACOLoader — 圧縮モデルを開く

Draco (Google製メッシュ圧縮) で圧縮された glTF を展開するデコーダを渡す

```
import { DRACOLoader } from
  'three/addons/loaders/DRACOLoader.js';

const draco = new DRACOLoader();
draco.setDecoderPath('/draco/');
loader.setDRACOLoader(draco);
// GLTFLoader にデコーダを接続
```

setDecoderPath

デコーダを配信する

setDRACOLoader

loader に接続する

KTX2Loader

GPU向け圧縮テクスチャ形式

PART 02

モデルを扱う

読み込んだモデルの調整と最適化を学ぶ

読み込み = Loader × Manager × Resource

Loader

取得

ファイルを解釈して
Object3D に変換する

×

Manager

管理

進捗・完了・失敗を
まとめて受け取る

×

Resource

資産

モデル・テクスチャ・
環境マップを扱う

3つがそろって初めて、外部の資産がシーンで動き出す

glTF と GLB を使い分ける

glTF (.gltf)

用途

中身を確認・編集したいとき

構成

.gltf + .bin + テクスチャ

特徴

人が読めるテキスト形式

GLB (.glb)

用途

配信・本番で扱いたいとき

構成

すべてを1つに同梱

特徴

転送が速く扱いやすい

開発中は中身を追える glTF、配信は1ファイルの GLB が扱いやすい

モデルを整える：中心とスケール

読み込んだモデルを囲って測り、原点とサイズをそろえる

Box3 で測る

setFromObject でモデルを囲む



中心をそろえる

getCenter で原点へ寄せる



大きさをそろえる

getSize から倍率を決める

```
const box = new THREE.Box3().setFromObject(model);  
const center = box.getCenter(new THREE.Vector3());  
model.position.sub(center); // 原点に寄せる  
const size = box.getSize(new THREE.Vector3()); // 大きさ
```

最小コードでモデル読み込み

```
import * as THREE from 'three';
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';

const loader = new GLTFLoader();
loader.load('/models/robot.glb', (gltf) => {
  const model = gltf.scene;
  const box = new THREE.Box3().setFromObject(model);
  model.position.sub(box.getCenter(new THREE.Vector3()));
  scene.add(model);
});

(function loop(){ controls.update();
  renderer.render(scene, camera); requestAnimationFrame(loop); })();
```

読み込みの流れ



読み込みは非同期。追加や調整は必ず onLoad の中で行う

アニメーションを再生する

glTF に同梱されたクリップを AnimationMixer で動かす

```
const mixer =  
  new THREE.AnimationMixer(glTF.scene);  
const clip = glTF.animations[0];  
mixer.clipAction(clip).play(); // 再生  
  
// animate 内で毎フレーム  
mixer.update(clock.getDelta());
```

glTF.animations

同梱クリップの配列

clipAction().play()

クリップを再生する

mixer.update()

毎フレーム時間を送る

軽く・速く読み込む

圧縮・破棄・使い回しで転送量とメモリを抑える

```
draco.setDecoderPath('/draco/'); // メッシュを圧縮
ktx2.setTranscoderPath('/basis/'); // テクスチャを圧縮
model.clone(); // 同じモデルを使い回す
geometry.dispose(); texture.dispose(); // 破棄
```

重い・時間がかかるときは？

Draco / KTX2 で圧縮し、使い終えた資産は dispose する。同じモデルは clone で使い回して読み込みを減らす。

よくあるつまずきポイント

Q モデルが見えない

A スケールと position、カメラの near / far を確認する

Q 真っ黒に映る

A ライトが environment（環境マップ）を追加する

Q 読み込めない

A パスと拡張子、ファイルの配信先を見直す

Q ローカルで CORS（通信制限）エラー

A 開発サーバー経由で配信し file:// を避ける

モデルを読み込んで並べよう

Step 1

モデルを用意

.glb を配置する

Step 2

GLTFLoader

load で読み込む

Step 3

中心・スケール

Box3 で調整する

Step 4

シーンに追加

add で複数配置

Step 5

操作と反応

Raycaster で選択

ゴール: 読み込んだモデルが整った状態で並び、クリックで色が変わって反応する

まとめ & 次回予告

今日のまとめ

- ✓ GLTFLoader で glTF / GLB を読み込む
- ✓ gltf.scene をシーンに追加する
- ✓ Box3 で中心とスケールを整える
- ✓ LoadingManager で進捗を管理する
- ✓ Draco / KTX2 で軽量化する

次回: SESSION 07

アニメーションと動きの演出

キーフレームと補間で動きを設計します。

- AnimationMixer でクリップを再生
- Clock と補間でなめらかに
- 状態に応じて動きを切り替え

用語集 — 今回登場した専門用語

本セッションに登場した専門用語をまとめて確認する

glTF

3Dモデルの標準形式

GLB

1ファイル形式

GLTFLoader

読み込みクラス

LoadingManager

進捗管理クラス

DRACOLoader

Draco展開クラス

KTX2Loader

テクスチャ展開クラス

Box3

範囲を測るクラス

AnimationMixer

アニメ再生クラス

dispose()

資産を解放するメソッド

clone()

複製して再利用

CORS

オリジン間通信の制限

NDC

正規化した座標値